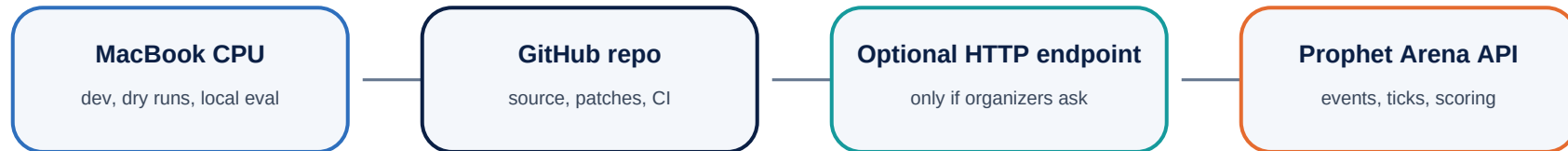


Forecasting Agent Monitoring Playbook

Operational architecture, dashboards, smoke tests, toy examples, and Kalshi-informed trading rules

Default: local-first. Host only when the rules require endpoint collection.



No GPU needed for API-based forecasting. No MCP/SSH required for the first working system.

Default decision: run local-first from the MacBook using API calls and the Prophet CLI/Core SDK. GitHub stores code. A hosted HTTP endpoint is optional, not the first dependency. A GPU box is only useful for local model inference or batch experiments, not for the baseline.

Question	Default answer	Why
Can this run from MacBook CPU?	Yes, for API-based forecasting, offline eval, traces, dashboards, and packaging.	The heavy compute is on model providers. Your laptop coordinates calls and writes logs.
Need a webserver now?	No. Use local module first. Add FastAPI only if organizers request endpoint collection.	Developer docs allow local module or HTTP endpoint; endpoint is only needed for organizer-side calls.
Need MCP server?	No for baseline. Optional if official trade path asks for MCP or if we want tool orchestration later.	MCP adds moving parts. We need fewer moving parts first.
Need SSH to another MacBook Pro?	No. Not unless we want a dedicated always-on runner.	Better: one clean repo, one command, logs. Avoid multi-machine confusion.
Need RunPod/GPU?	Not initially.	Use only for local vLLM/batch inference experiments after the agent is stable.

Success before live: clean dry runs, complete traces, offline Brier/ECE vs baselines, no rule violations, no unbounded model spend, and a run summary you can understand in under five minutes.

1. The Monitoring Funnel

Monitor the funnel, not the vibes



If a stage drops unexpectedly, the next action is obvious: fix filter, forecast, retrieval, edge, risk, or API.

Signal	Green	Yellow	Red / fix immediately
Trace completeness	100% decisions logged	Optional fields missing	p_yes, edge, action, or skip reason missing
JSON parse failures	0%	< 2%	> 2% or any uncaught exception
Model spend	Under cap	Near cap	Unbounded calls or no per-call cost log
Forecast distribution	Varied, sane, mostly 0.1-0.9	Too many 0.50s	Extreme values without evidence
Market deviation	Explained and logged	Large but plausible	Raw model overrides market with no evidence
Retrieval	Few high-quality sources	Neutral benefit	Many noisy sources, worse score
Trading	Skipped unless edge clears	Minor rejects	Rejected intents or risk violation

The dashboard should show the funnel, not just final PnL. A bad run should tell us whether the failure came from filtering, forecasting, source quality, calibration, edge calculation, risk, API timing, or packaging.

2. Smoke Test Ladder

Level	Test	Command shape	Pass condition
0	Import/config/rules	python tools/check_ruleset.py && python tools/validate_config.py	No secret leaks; risk constants <= official limits
1	Dry run startup	python agent.py --slug smoke --dry-run	Creates log/trace dirs; prints config hash; exits 0
2	Toy monitoring harness	python monitoring_demo.py --events toy_events.jsonl --out outputs	JSONL trace + Markdown summary + charts created
3	Offline dataset forecast	prophet forecast retrieve -o events.json; prophet forecast predict --events events.json --local my_agent	Every event returns p_yes in [0.01, 0.99]
4	Local scoring	prophet forecast evaluate --submission predictions.json --actuals actuals.json	Brier score computed; variant compared to baseline
5	Live test slug	python agent.py --slug test-live-v1	Claim, load, submit/finalize/complete works
6	Submission packaging	python tools/package_submission.py --dry-run	Zip excludes .env, .git, venv, caches, bulky logs

Operating rule: do not add agents, retrieval, or RL-lite until Levels 0-2 are green. Complexity before traceability is the easiest way to get lost.

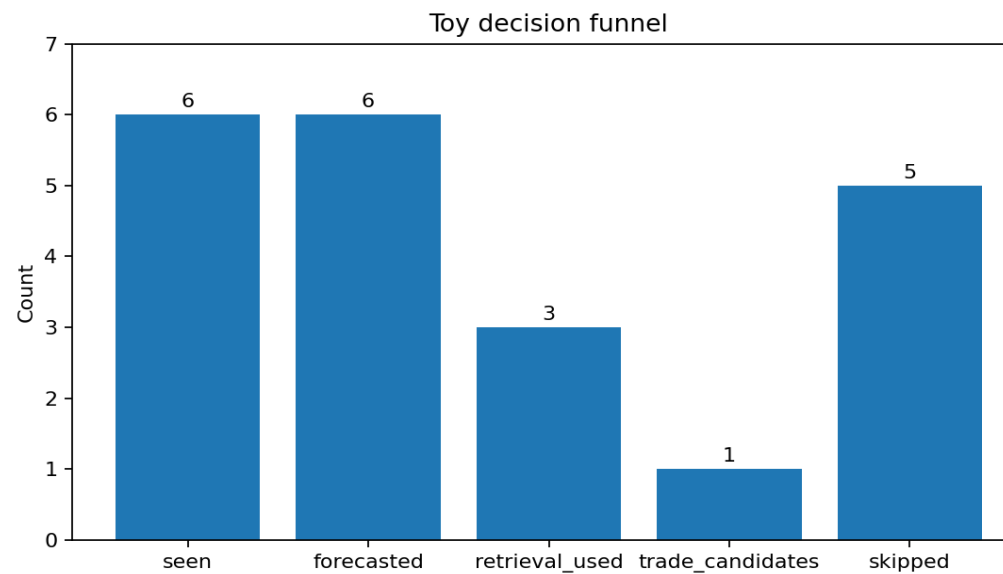
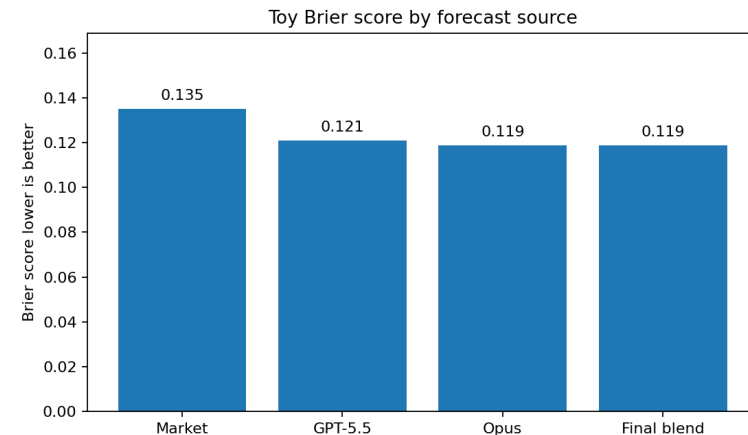
Minimum daily status note:

date, default variant, last green command, current blocker, best offline Brier, market-only Brier, ECE, trace completeness, parse failure rate, cost per 100 forecasts, known red risks, next action.

3. Toy Example: One Offline Monitoring Run

The attached demo zip includes six toy markets. It blends market prior + GPT-5.5 + Opus-style forecasts, applies a Kalshi-inspired longshot guard, computes executable edge, and writes a JSONL trace. It is tiny by design: the goal is to test observability, not claim statistical validity.

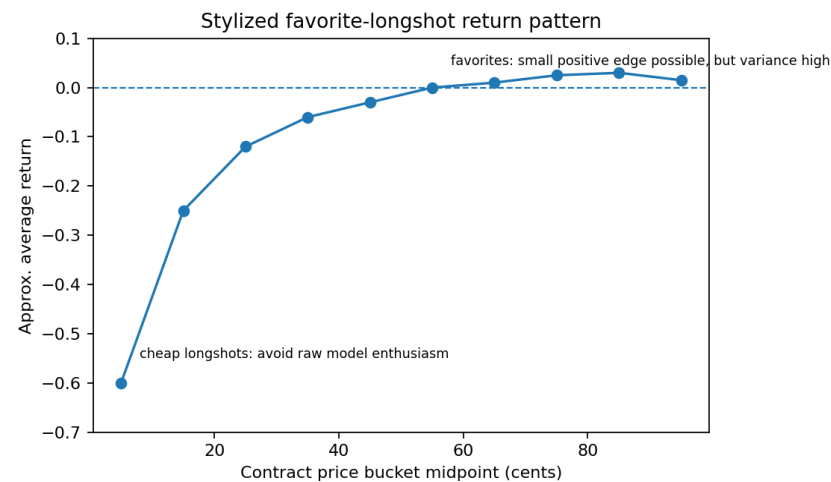
Metric	Market	GPT-5.5	Opus	Final blend
Mean Brier	0.1352	0.1210	0.1187	0.1188
ECE	0.1600	0.1800	0.1567	0.1475
BUY decisions	-	-	-	1 of 6
SKIP decisions	-	-	-	5 of 6



Read the toy result like this: the final blend is close to the best model, improves ECE relative to raw GPT-5.5, and trades only one high-evidence weather case. That is the shape we want: measured, logged, and conservative.

4. More Realistic Example: Kalshi-Informed Market Behavior

The Kalshi paper matters because it shows prediction-market prices are informative but not unbiased. It documents a favorite-longshot bias: cheap contracts win less often than their prices imply, while high-price contracts tend to do better and can earn small positive returns. It also finds Makers outperform Takers, but both still show the longshot pattern.



Kalshi takeaway	Agent rule
Low-price contracts underperform	Add longshot guard below ~10c. Do not let LLM chatter pull 0.07 to 0.20 without primary evidence.
High-price contracts may be slightly underpriced	Do not over-shrink strong favorites just because LLMs are conservative.
Makers beat Takers	In the benchmark we mostly submit deterministic intents, but spread/execution still matter. Avoid paying spread on weak edge.
Prices improve near close	Near resolution, markets may update faster than model/retrieval. Require fresher evidence.

Practical implementation: compute p_{market} from executable prices; compute p_{model} from model ensemble; apply source-quality and longshot/favorite guards; trade only if executable edge clears the risk-adjusted threshold.

5. Model Mix: Not LLM-Only

Not LLM-only: use an ensemble of priors, models, retrieval, calibration, and hard rules



A frontier model can reason, but code owns the probability hygiene, blending, thresholds, and position sizing.

Component	Role	What to monitor
Market prior	Strong baseline and crowd signal	Brier vs model; deviation size; near-resolution behavior
GPT-5.5 / Opus	Reasoning, synthesis, uncertainty	JSON parse rate; source use; p_yes distribution; cost/latency
Model averaging	Reduce individual model variance	Pairwise disagreement; ensemble improvement vs single model
Calibration	Shrink overconfidence and longshot errors	ECE, reliability chart, bucket outcomes
Rules/statistics	Hard constraints and domain logic	Rule violations, skip reasons, edge threshold
Retrieval	Fresh primary evidence when useful	Source type, recency, credibility, whether Brier improves

LeCun-style caveat accepted: an LLM is not a complete world model. That is why the submission should combine markets, calibration, domain rules, retrieval, and model averaging. LLMs reason; the system measures, constrains, and corrects them.

6. Build Checklist: What We Should Add Next

Priority	Artifact	Purpose
1	tools/monitor_run.py	Read trace files and print funnel, parse failures, cost, skip reasons, rule warnings.
2	tools/evaluate_offline.py	Compare market-only, GPT-only, Opus-only, blend, retrieval, and ensemble variants.
3	tools/check_compliance.py	Assert no secrets, pinned versions, risk caps <= official rules, clean package.
4	forecasting/market_blend.py	Credibility-weighted logit blend with longshot/favorite guards.
5	forecasting/ensemble.py	Median/logit pooling and disagreement-based abstention.
6	reports/status.md	Human-readable operating status so you are never guessing progress.
7	FastAPI endpoint	Only if organizer-run endpoint collection is required. Local module first.

Default build path: local monitoring harness -> local forecast module -> offline eval -> model blend -> endpoint only if requested -> live test slug. This keeps the project understandable and keeps infrastructure from becoming the project.

Source notes

This playbook is grounded in: Prophet Arena/LLM-as-a-Prophet pipeline and metrics; Kalshi Makers and Takers favorite-longshot/microstructure findings; the Prophet Hacks operating pack, baseline strategy, budget policy, research spine, and submission checklist; and the current AI Prophet developer/build-a-bot docs.